3️⃣ Indexing & Slicing (Very Important)

- 1D, 2D, 3D slicing
- Negative indexing
- Step slicing
- Boolean indexing
- Fancy indexing (ADVANCED)
- Multidimensional selection tricks

⚠️ W3Schools does NOT deeply cover:

- Broadcasting inside indexing
- Combining boolean + fancy indexing

🔵 INDEXING & SLICING (Deep + Complete)

1️⃣ 1D Indexing

```
arr = np.array([10,20,30,40,50])


Basic indexing

arr[0]     # 10

arr[-1]    # 50


Slicing

arr[1:4]   # 20,30,40

arr[:3]

arr[2:]
```

⚠ Slicing returns view, not copy.

2️⃣ Step Slicing

```
arr[::2]    # every 2nd element
arr[::-1]   # reverse
arr[1:5:2]
```

Format:

```
[start : stop : step]
```

3️⃣ 2D Indexing

```
arr = np.array([[1,2,3],
                [4,5,6]])
```

Access element

```
arr[0,1]    # 2
```

Row

```
arr[0]
```

Column

```
arr[:,1]
```

Submatrix

```
arr[0:2, 1:3]
```

Format:

```
arr[row_selection, column_selection]
```

4️⃣ 3D Indexing

```
arr = np.array([
  [[1,2],[3,4]],
  [[5,6],[7,8]]
])
```

Shape:

```
(2,2,2)
```

Access element:

arr[0,1,1]

Slice:

arr[:, :, 0]

General format:

arr[axis0, axis1, axis2]

5️⃣ Boolean Indexing (Very Important)

arr = np.array([10,20,30,40])

arr[arr > 20]

Returns:

[30,40]

Works element-wise.

With 2D:

arr = np.array([[1,2],[3,4]])

arr[arr > 2]

Returns flattened result:

[3,4]

6️⃣ Fancy Indexing (ADVANCED)

Index using list/array of indices.

arr = np.array([10,20,30,40])

arr[[0,3]]

Returns:

[10,40]

2D Fancy

arr = np.array([[1,2],[3,4]])

arr[[0,1], [1,0]]

Selects:

(0,1) and (1,0)

Result:

[2,3]

⚠ Fancy indexing returns COPY (not view).

7️⃣ Boolean + Fancy Combined (Important)

arr = np.array([10,20,30,40])

mask = arr > 20

arr[mask]

You can also combine:

arr[(arr > 10) & (arr < 40)]

Use:

- & not and
- | not or

8️⃣ Broadcasting Inside Indexing

Example:

arr = np.arange(9).reshape(3,3)

Select rows with array:

rows = np.array([0,2])

arr[rows]

Column selection with broadcasting:

arr[:, [0,2]]

Row + column selection:

arr[[0,1], :]

9️⃣ **Multidimensional Selection Tricks**

Select specific rows and columns grid

arr[np.ix_([0,2], [1,2])]

This creates proper 2D selection.

Without ix_, broadcasting rules may surprise you.

✅ **When to use np.ix_()**

Use np.ix_() **only when**:

1. You want to **select multiple rows AND multiple columns at the same time**
   → And you want a **full 2D submatrix (grid)**, not just paired elements.
2. Your row indices array and column indices array have **different lengths** or shapes.
   → This avoids broadcasting surprises.

1️⃣1️⃣ Modify Using Indexing

arr[arr > 20] = 0

Works in-place.

1️⃣2️⃣ Ellipsis (...)

Shortcut for full dimensions.

arr[..., 0]

Useful in high dimensions.

Select elements at (0,1), (1,2), (2,3).

[[0 1 2 3]

 [4 5 6 7]

 [8 9 10 11]

 [12 13 14 15]]

print(ar[[0,1,2],[1,2,3]])

[1 6 11]